

Reporte de Pruebas — Postman

Sistema loteur — Validación de endpoints REST

Configuración del Entorno

Las pruebas se ejecutaron levantando el entorno completo de la plataforma loTEUR mediante Docker Compose en modo desarrollo. Todos los servicios (API Gateway, Auth, Devices, Register, Telemetry, Notifications, Redis y bases de datos) se iniciaron simultáneamente y los logs se guardaron en un archivo para análisis posterior.

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build | Tee-Object -FilePath postman-errors.log
```

Fase 1 — Gestión de Usuarios

Se validaron los endpoints del servicio de autenticación para usuarios regulares. El flujo completo cubre el ciclo de vida de una sesión: registro, inicio de sesión, consulta del perfil, renovación del token de acceso, cierre de sesión y verificación de que el token queda invalidado tras el logout.

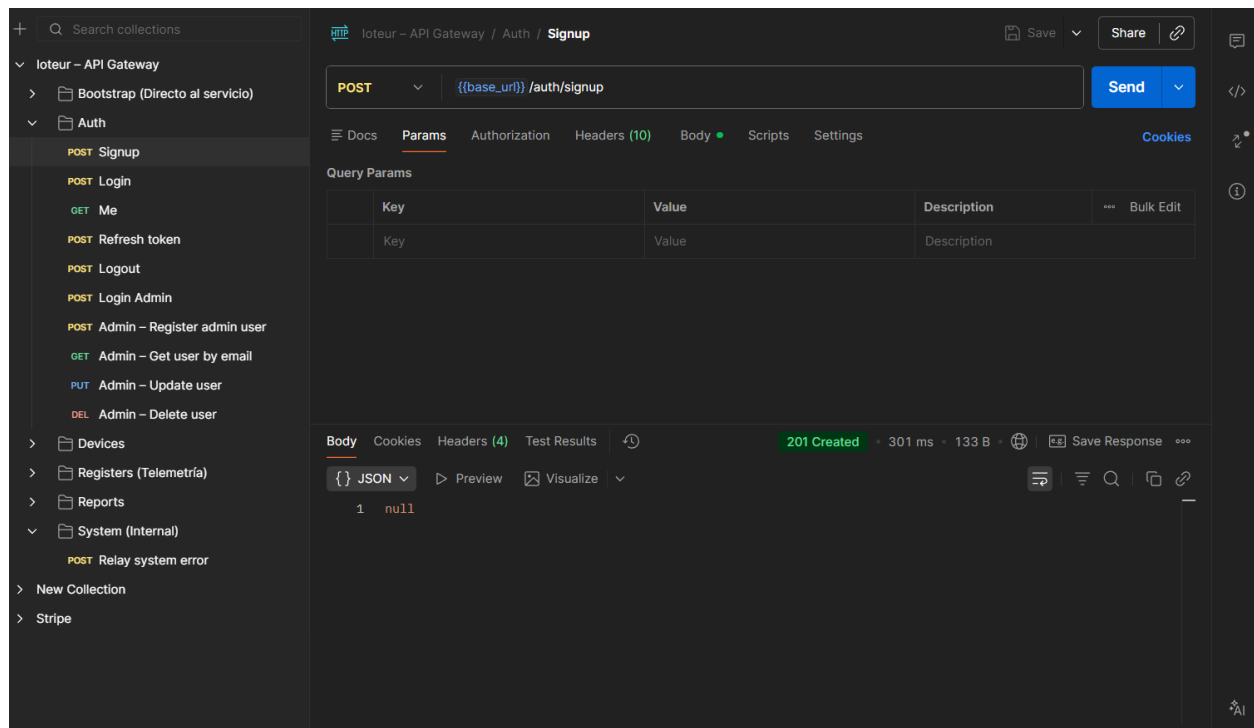


Figura 1 — Registro de usuario (POST /auth/signup)



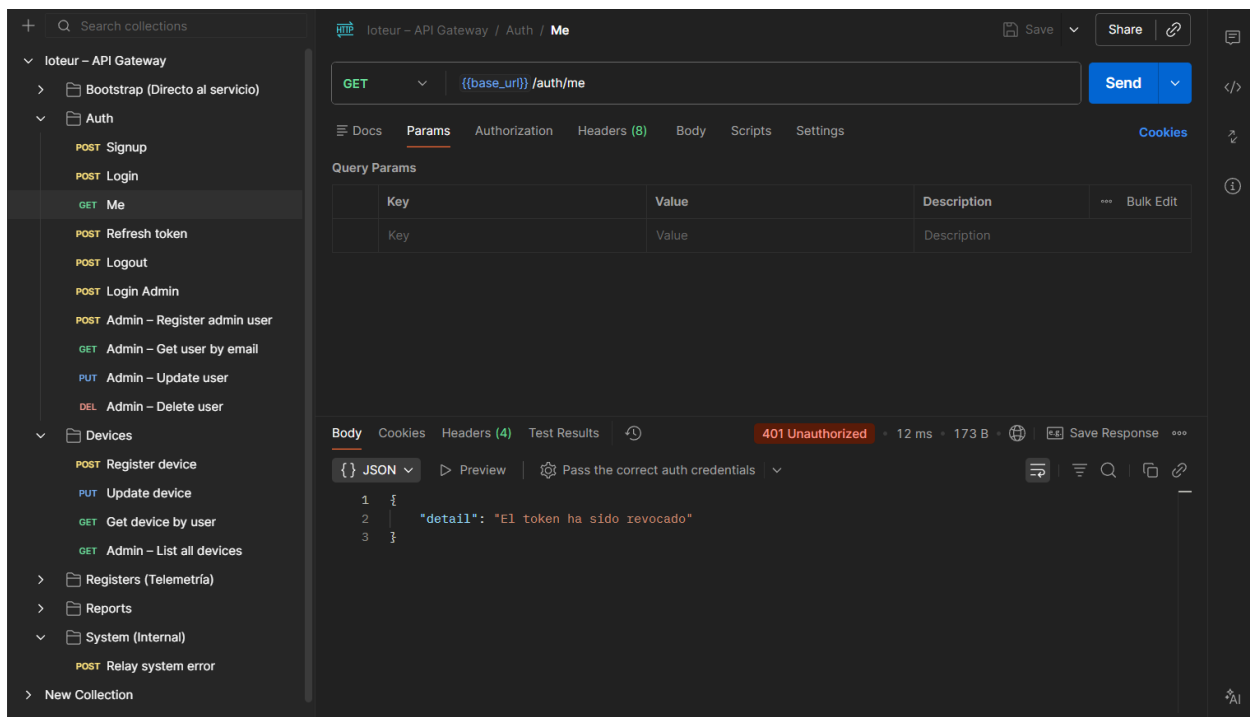


Figura 6 — Obtener perfil propio con token revocado (GET /auth/me)

Fase 2 — Administración

Se probaron los endpoints exclusivos del rol administrador. Estos requieren un token JWT con privilegios elevados y permiten registrar nuevos usuarios, autenticarse como admin, consultar usuarios por correo electrónico y modificar datos de cualquier cuenta del sistema.

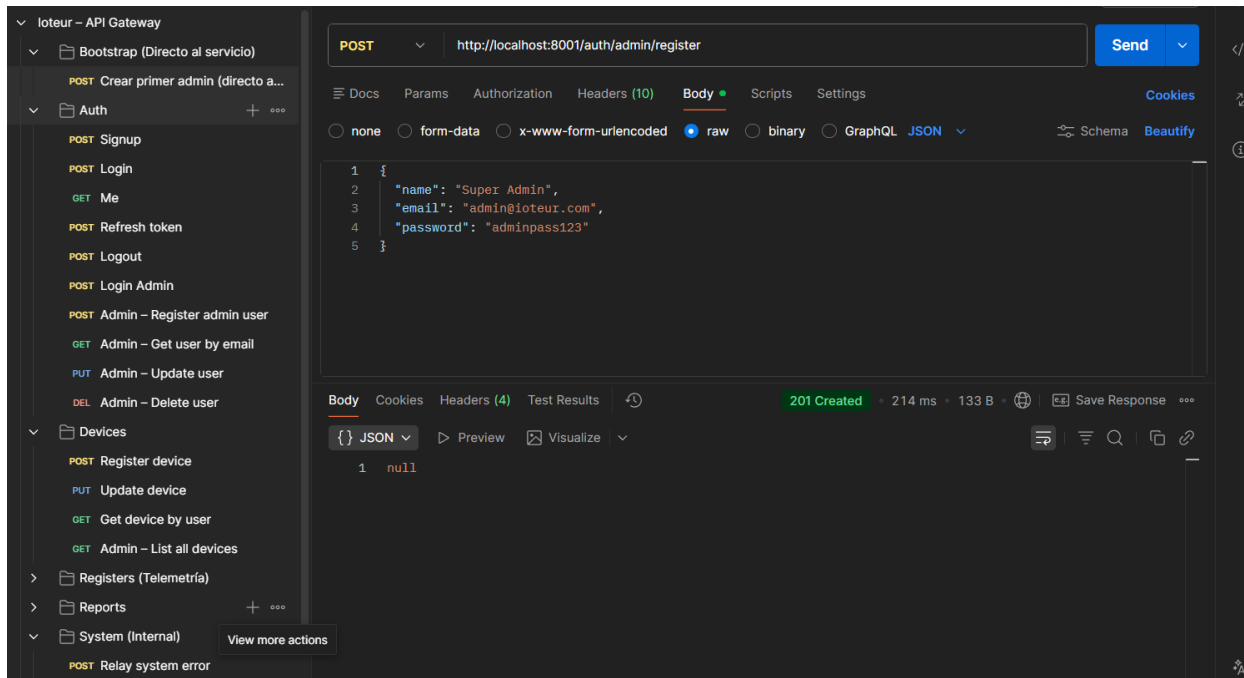


Figura 7 — Registro de usuario por admin (POST /auth/admin/register)

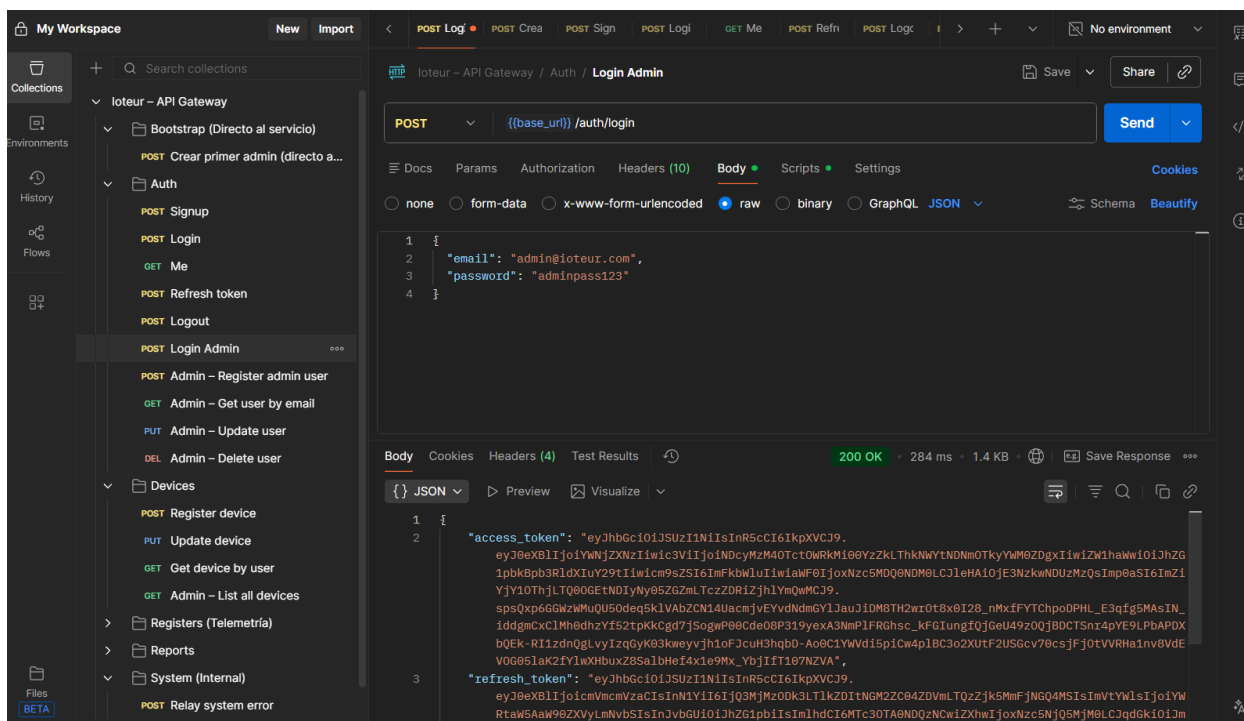


Figura 8 — Login de administrador (POST /auth/login)

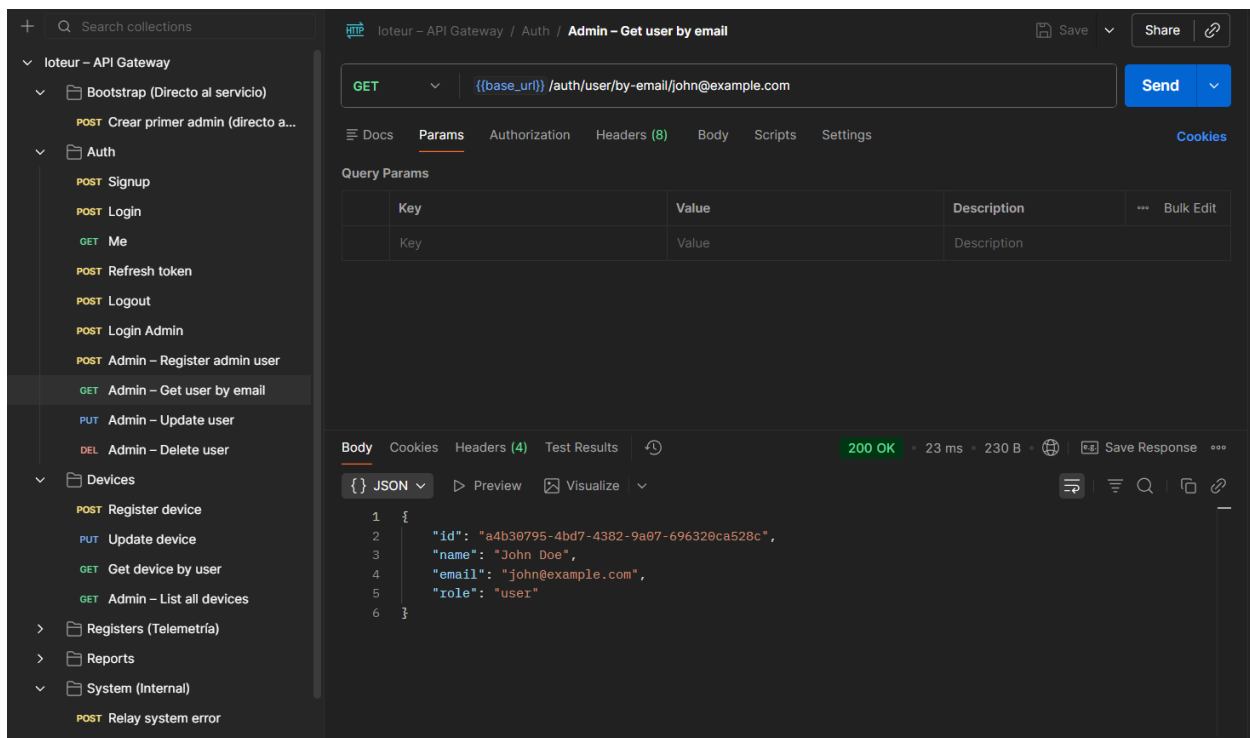


Figura 9 — Listar un usuario por email (GET /auth/users/by-email/:email)

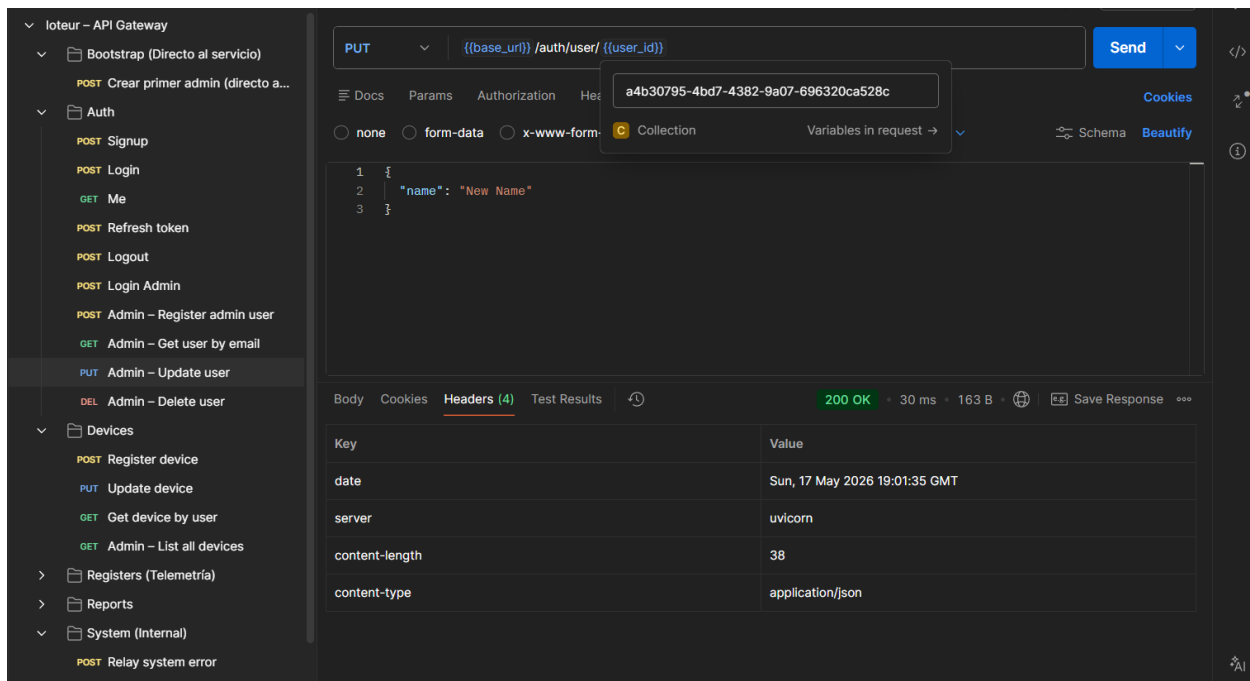


Figura 10 — Modificar un usuario específico (PUT /auth/user/:id)

Fase 3 — Gestión de Dispositivos

Se validaron los endpoints del servicio de dispositivos. Las pruebas cubren el registro de un nuevo dispositivo asociado a un usuario, la consulta del listado de dispositivos propios, la actualización de sus atributos y la consulta de dispositivos por un admin. Los dispositivos se persisten en PostgreSQL y se cachean en Redis.

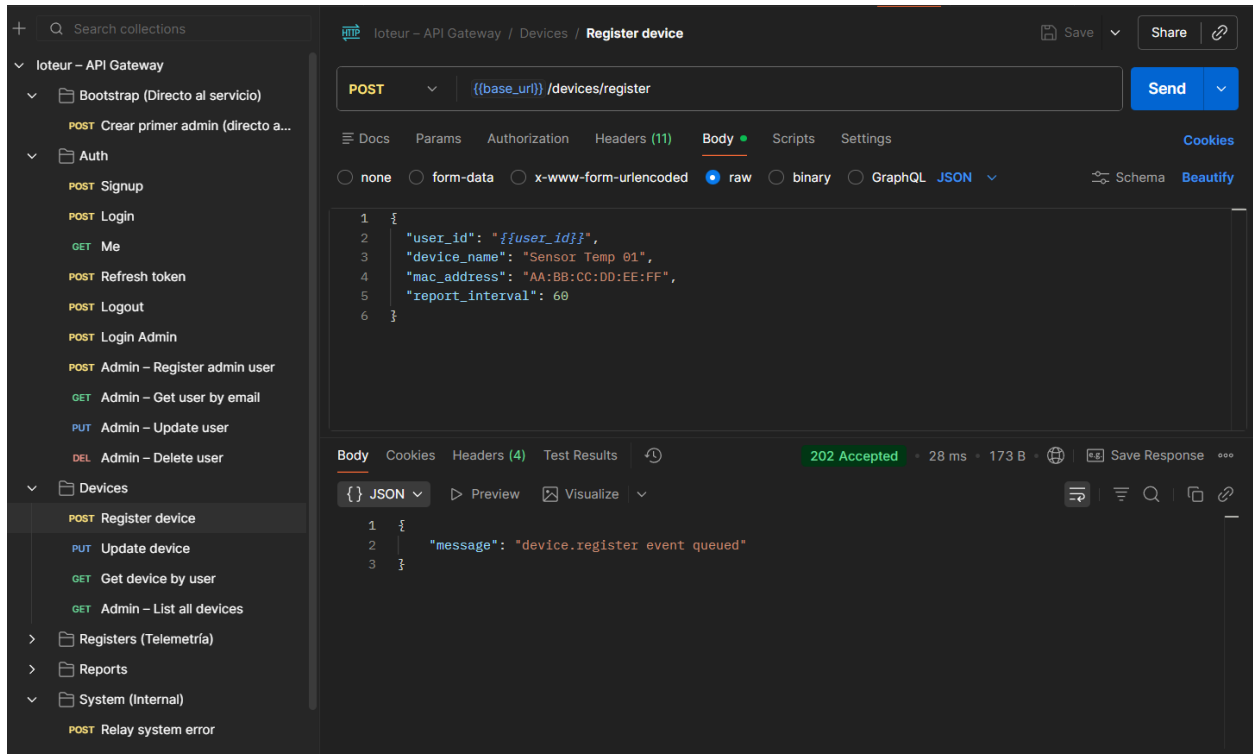


Figura 11 — Registrar dispositivo (POST /devices/register)

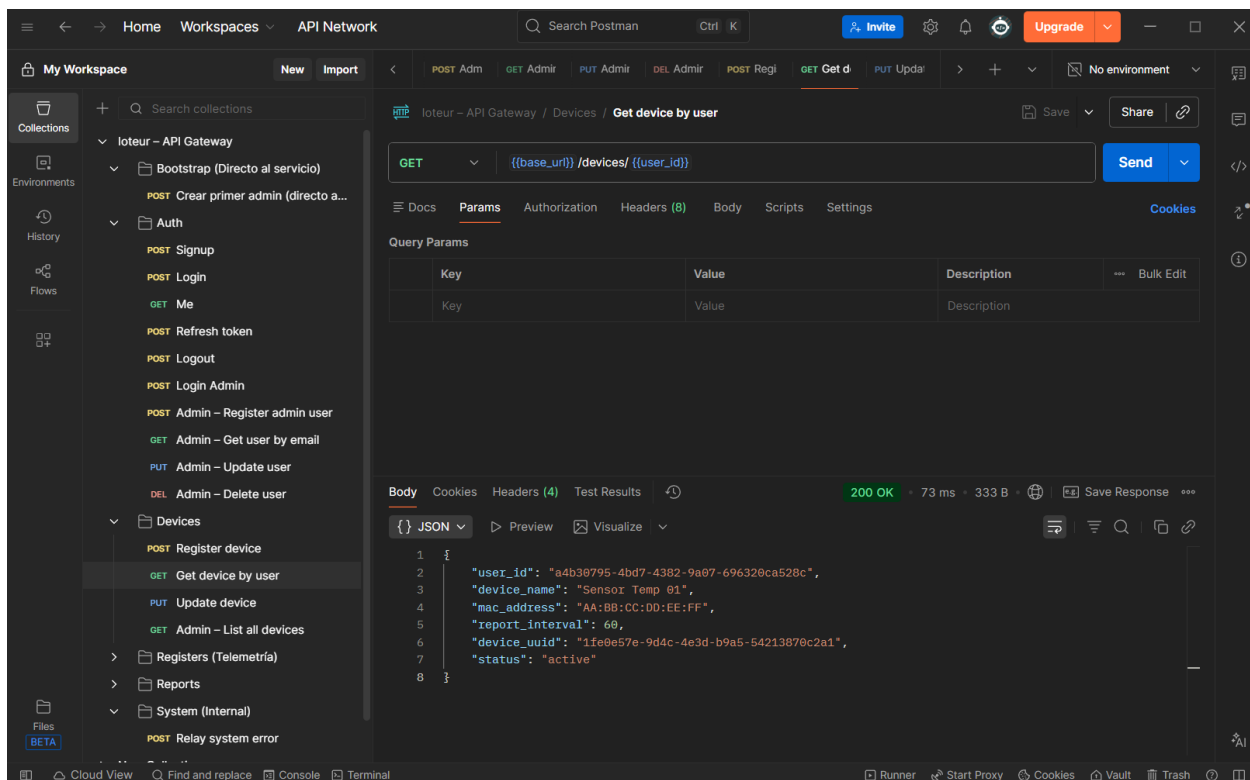


Figura 12 — Listar dispositivos del usuario (GET /devices/{user_id})

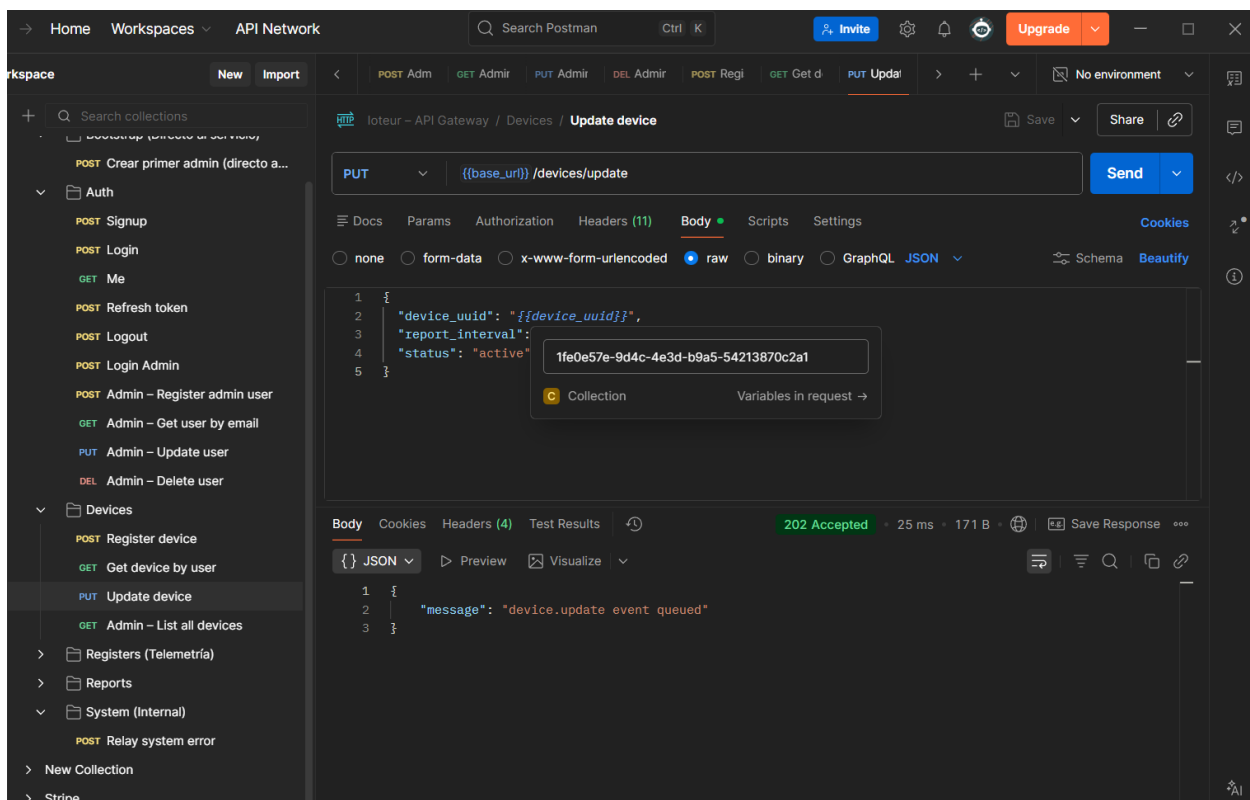


Figura 13 — Actualizar un dispositivo por id (PUT /devices/update)

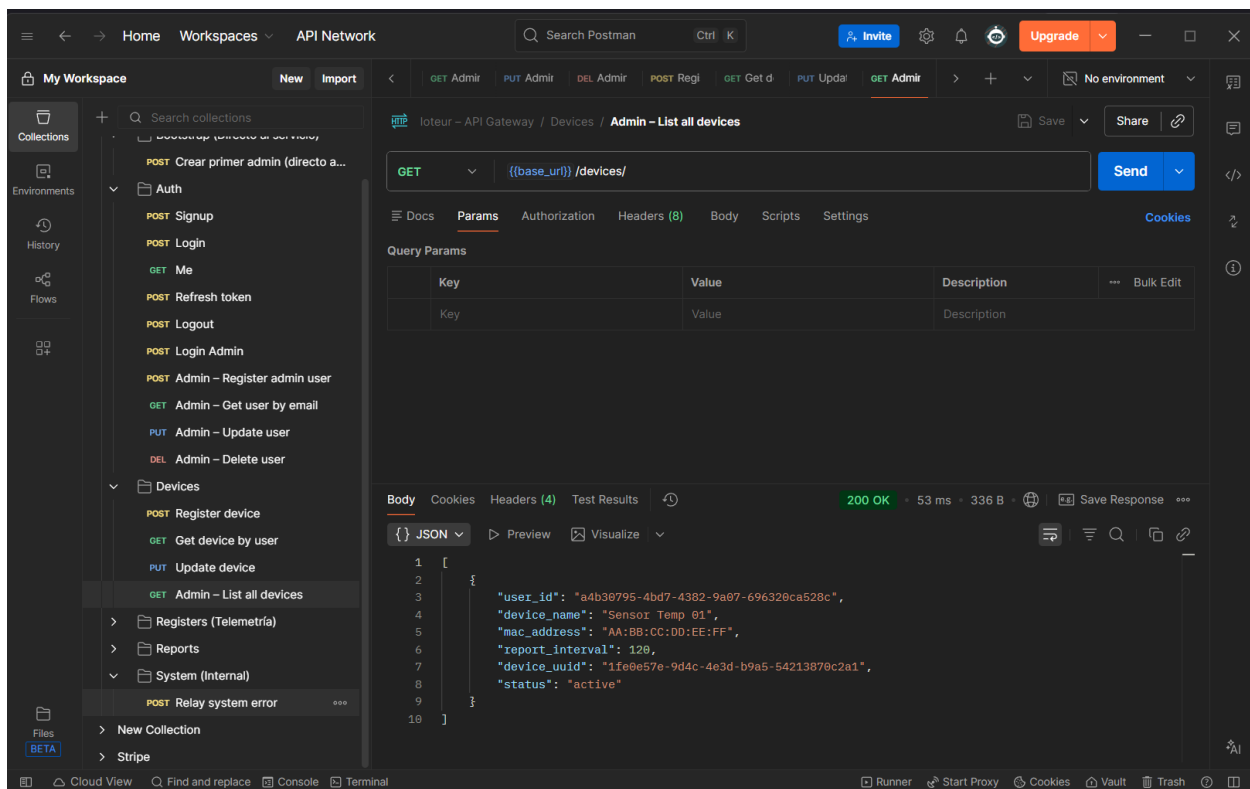


Figura 14 — Listar todos los dispositivos Admin (GET /devices/)

Fase 4 — Gestión de Registros

Se verificó el flujo de ingesta de datos de telemetría. Un dispositivo envía una lectura (temperatura y humedad) al endpoint de recepción, el sistema la procesa y la almacena en MongoDB. Posteriormente se consultan los registros históricos del dispositivo y se prueba la integración con el módulo de reportes, incluyendo un escenario con rango de fechas del año 2025.

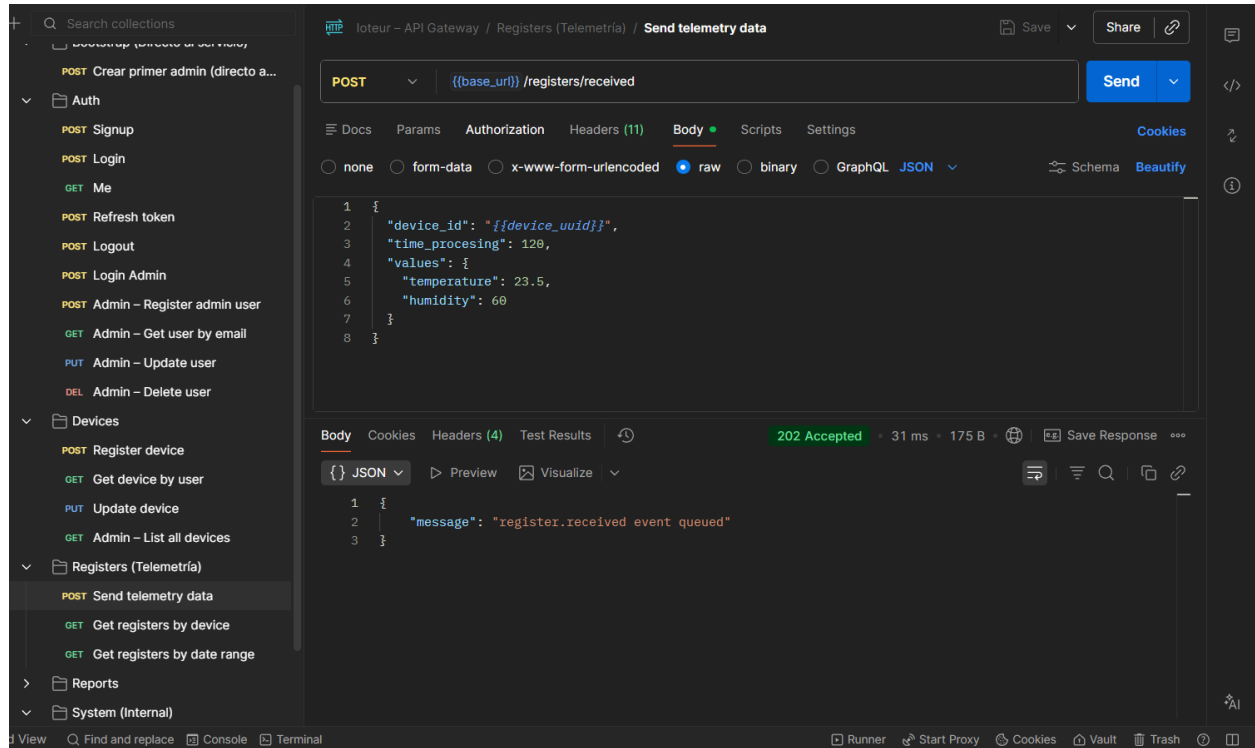


Figura 15 — Enviar un registro (POST /registers/received)

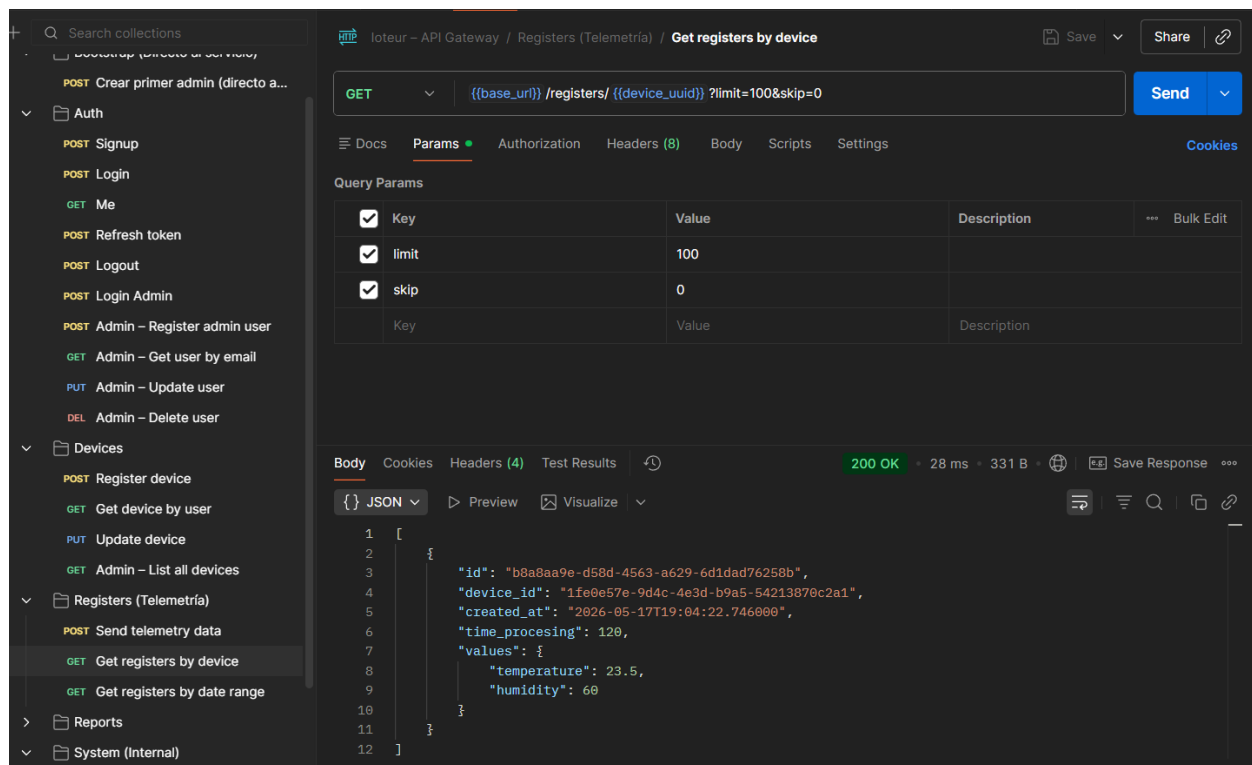


Figura 16 — Listar registros de un dispositivo (GET /registers/:device_uuid)

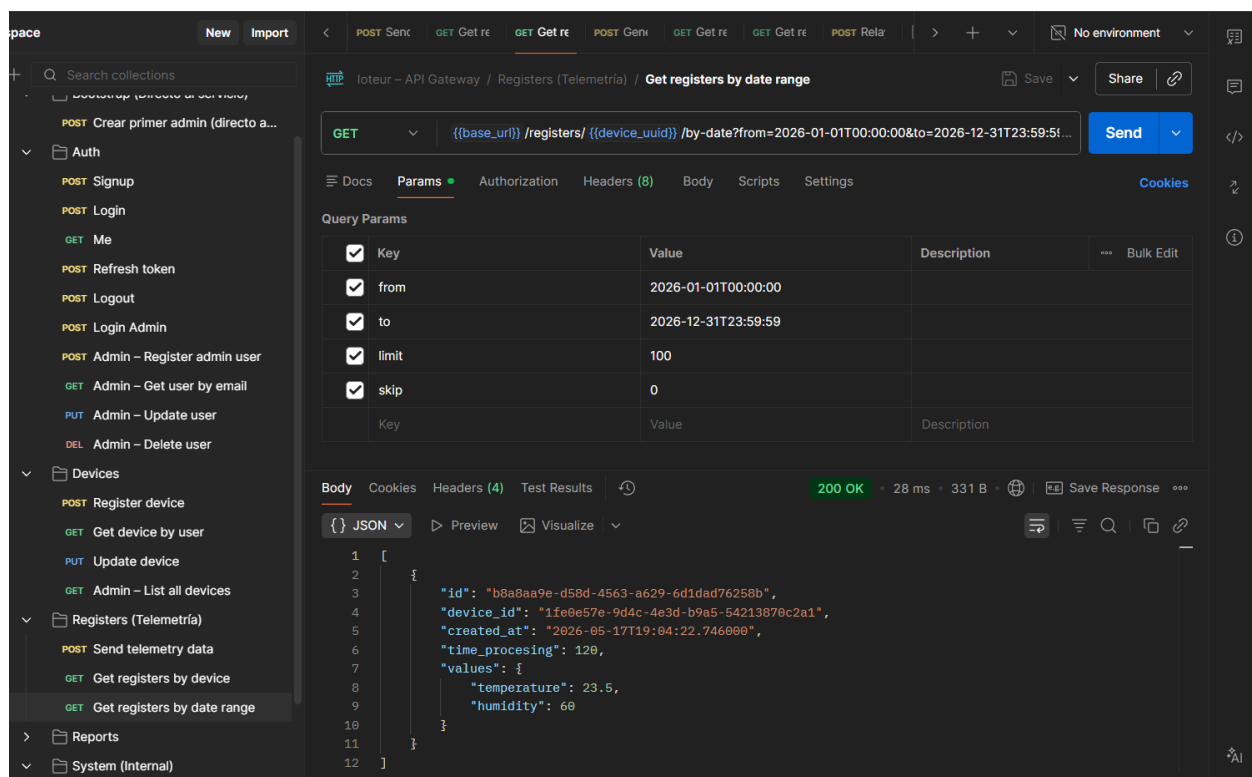


Figura 17 — Reporte de registros con fechas 2026 (GET /reports/by-date)

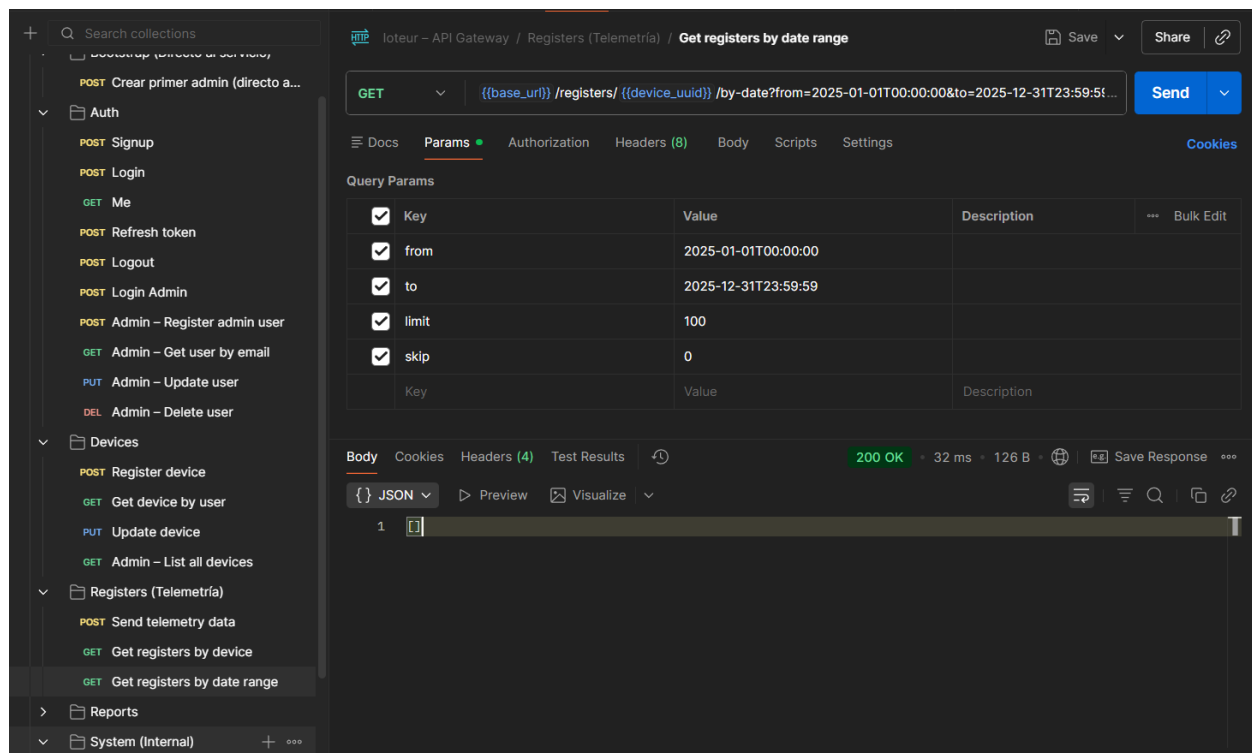


Figura 18 — Reporte de registros con fechas 2025 (GET /reports/by-date)

Fase 5 — Reportes de Telemetría y Registro de Error del Sistema

Se validaron los endpoints del servicio de reportes de telemetría. Se generó un reporte para el dispositivo activo, se consultaron reportes filtrando por dispositivo específico, por rango de fechas del año 2026 con datos existentes, y por un rango de fechas sin registros (respuesta vacía esperada). Adicionalmente, se provocó y verificó el correcto registro de una notificación de error crítico del sistema.

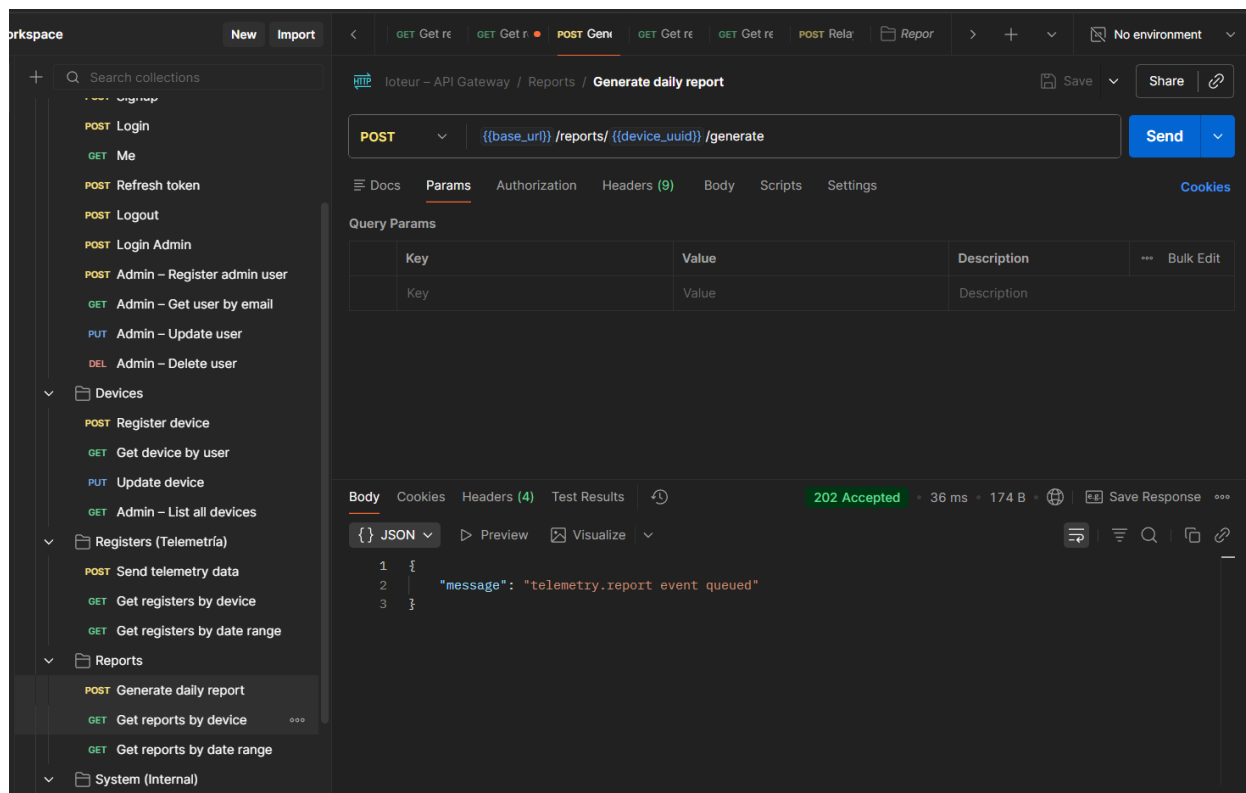


Figura 19 — Generar reporte de telemetría (POST /reports/generate)

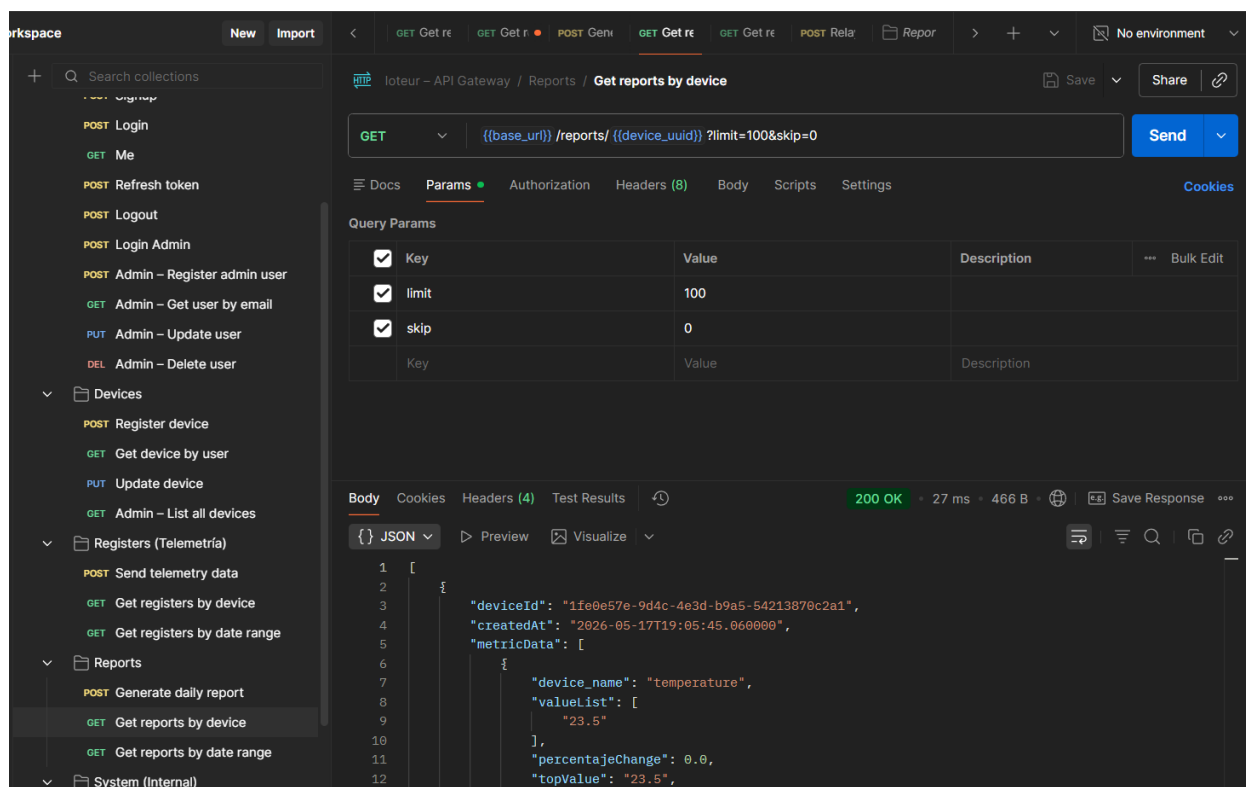


Figura 20 — Búsqueda de reportes para un dispositivo específico (GET /reports/:device_id)

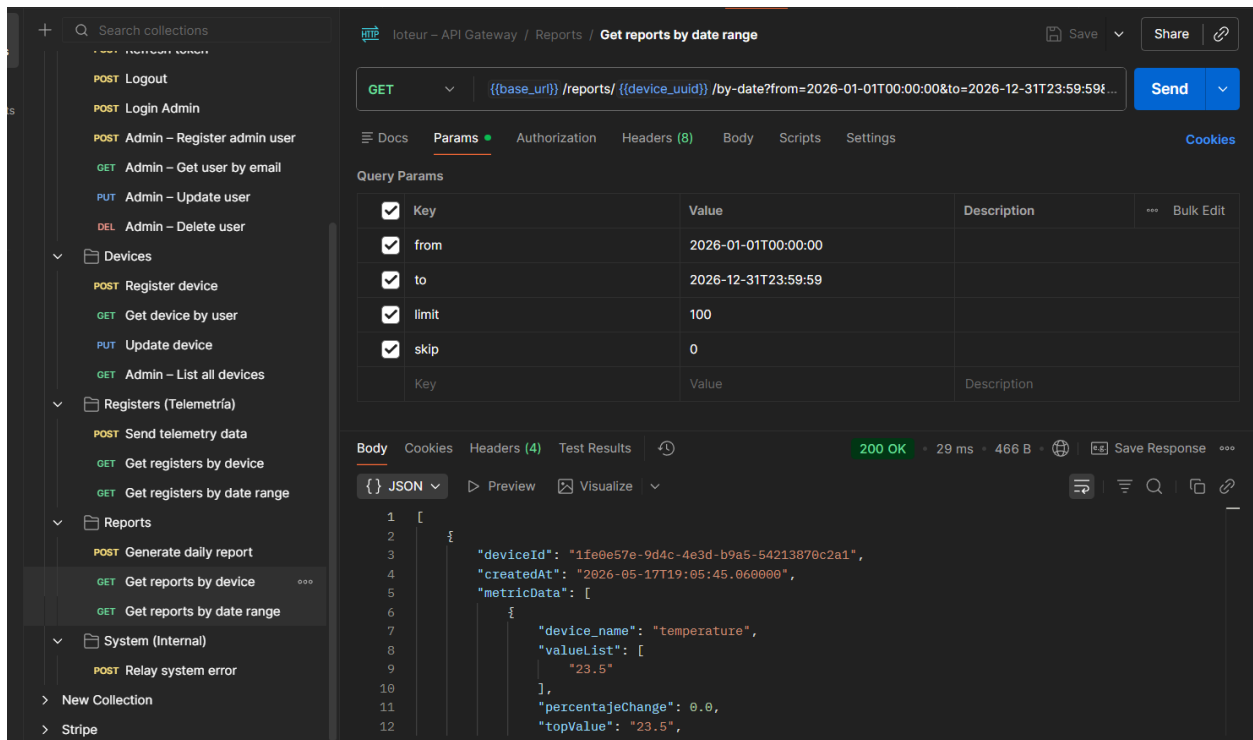


Figura 21 — Búsqueda de reportes en el año 2026 (GET /reports/by-date)

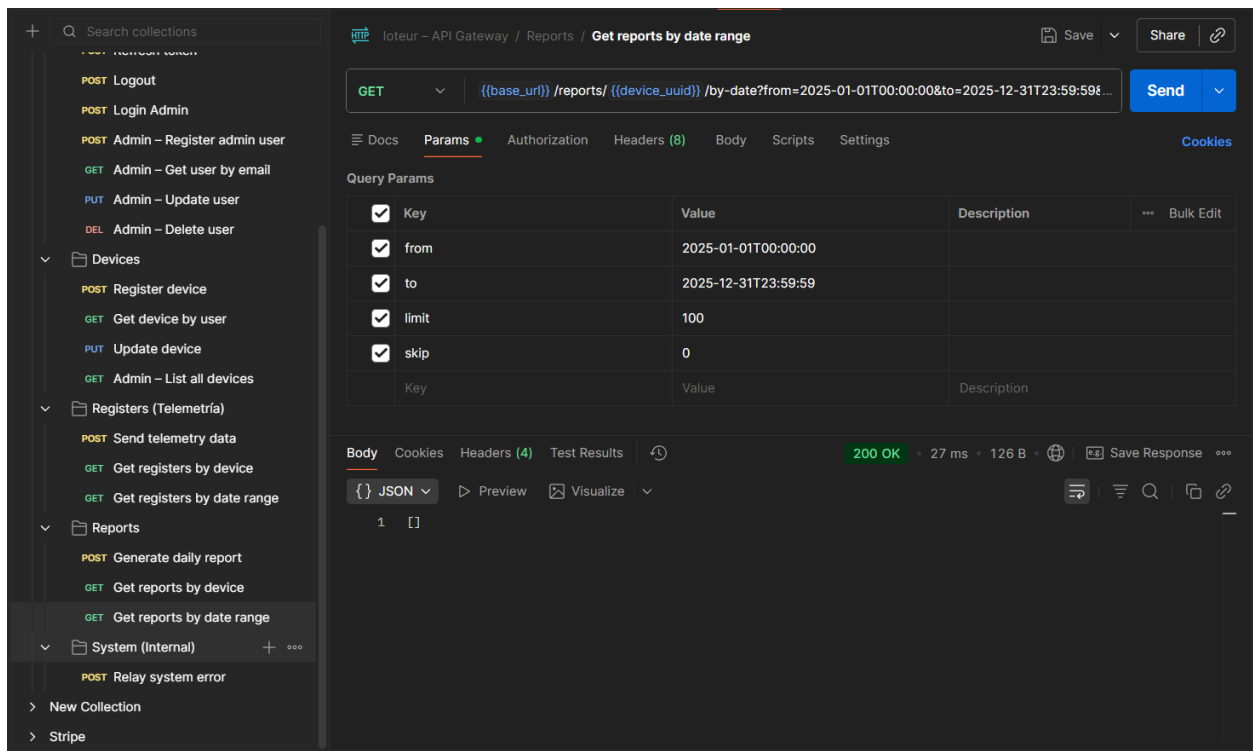


Figura 22 — Búsqueda de reportes en fechas sin registros (respuesta vacía)

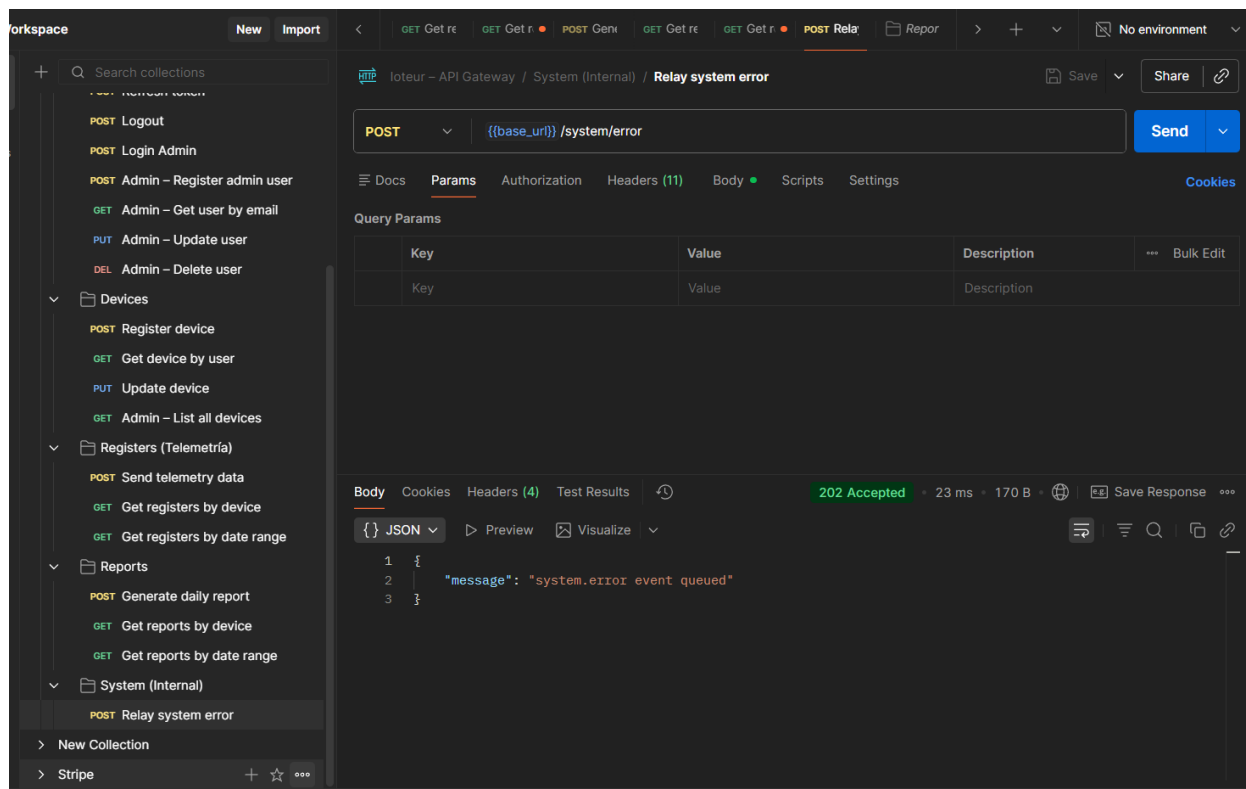


Figura 23 — Levantar un error del sistema (notificación crítica)

Fase 6 — Verificación de Registros en Bases de Datos

Concluidas las pruebas funcionales, se inspeccionaron directamente todas las bases de datos para confirmar la correcta persistencia de los datos en cada servicio. Se verificaron PostgreSQL (autenticación y dispositivos), MongoDB (registros, telemetría y notificaciones) y Redis (blacklist de tokens y caché de dispositivos).

6.1 PostgreSQL — Servicio de Autenticación

Se consultaron los usuarios registrados en auth_db. Se encontraron dos registros activos: el Super Admin del sistema y el usuario de prueba creado durante las pruebas.

```
docker exec -it ioteur-postgres-auth-1 psql -U auth_user -d auth_db -c "SELECT id, name, email, role FROM users;"
```

```
PS C:\Users\luigi> docker exec -it ioteur-postgres-auth-1 psql -U auth_user -d auth_db -c "SELECT id, name, email, role FROM users;"
id | name | email | role
---|---|---|---
47233897-9dd2-4c6d-8d5f-43f992ac4d81 | Super Admin | admin@ioteur.com | admin
a4b30795-4bd7-4382-9a07-696320ca528c | New Name | john@example.com | user
(2 rows)
```

Figura 24 — Registros en PostgreSQL auth_db (tabla users)

6.2 PostgreSQL — Servicio de Dispositivos

Se verificó que el dispositivo "Sensor Temp 01" quedó correctamente registrado y vinculado al UUID del usuario de prueba, con estado activo y MAC address asignada.

```
PS C:\Users\luigi> docker exec -it ioteur-postgres-device-1 psql -U device_user -d device_db -c "SELECT device_uuid, user_uuid, device_name, mac_address, status FROM devices;"
```

device_uuid	user_uuid	device_name	mac_address	status
1fe0e57e-9d4c-4e3d-b9a5-54213870c2a1	a4b30795-4bd7-4382-9a07-696320ca528c	Sensor Temp 01	AA:BB:CC:DD:EE:FF	active

(1 row)

Figura 25 — Registros en PostgreSQL device_db (tabla devices)

6.3 MongoDB — Registro de Telemetría

La base de datos register_db contiene el documento de ingesta generado durante la Fase 4, con los valores de temperatura (23.5°C) y humedad (60%), tiempo de procesamiento y timestamp UTC.

```
PS C:\Users\luigi> docker exec -it ioteur-mongo_register-1 mongosh "mongodb://register_user:register_pass@localhost:27017/register_db?authSource=admin" --eval "db.register.find().pretty()"
```

```
[
  {
    _id: 'b0a9a9e-d58d-4563-a629-6d1dad76250b',
    device_id: '1fe0e57e-9d4c-4e3d-b9a5-54213870c2a1',
    created_at: ISODate('2026-05-17T19:04:22.746Z'),
    time_processing: 120,
    values: { temperature: 23.5, humidity: 60 }
  }
]
```

Figura 26 — Documentos en MongoDB register_db

6.4 MongoDB — Reportes Diarios de Telemetría

La base de datos telemetry_db generó el reporte diario agregado con métricas por métrica: valor máximo, valor más frecuente, lista de valores y porcentaje de cambio para temperatura y humedad.

```
PS C:\Users\luigi> docker exec -it ioteur-mongo_telemetry-1 mongosh "mongodb://telemetry_user:telemetry_pass@localhost:27017/telemetry_db?authSource=admin" --eval "db.daily_reports.find().pretty()"
```

```
[
  {
    _id: ObjectId('6a0a11894b16817174969c16'),
    device_id: '1fe0e57e-9d4c-4e3d-b9a5-54213870c2a1',
    created_at: ISODate('2026-05-17T19:05:45.060Z'),
    metric_data: [
      {
        device_name: 'temperature',
        value_list: [ '23.5' ],
        porcentaje_change: 0,
        top_value: '23.5',
        most_frequent_value: '23.5'
      },
      {
        device_name: 'humidity',
        value_list: [ '60' ],
        porcentaje_change: 0,
        top_value: '60.0',
        most_frequent_value: '60'
      }
    ]
  }
]
```

Figura 27 — Documentos en MongoDB telemetry_db (daily_reports)

6.5 MongoDB — Notificaciones del Sistema

La base de datos notifications_db registró la notificación crítica generada en la Fase 5. Se puede observar el servicio de origen, la razón del fallo (db_connection_failed) y la severidad crítica.

```
PS C:\Users\luigi> docker exec -it ioteur-mongo_notifications-1 mongosh "mongodb://notifications_user:notifications_pass@localhost:27017/notifications_db?authSource=admin" --eval "db.getCollectionNames().forEach(c => { print(c); db[c].find().pretty() })"
```

```
system_notifications
PS C:\Users\luigi> docker exec -it ioteur-mongo_notifications-1 mongosh "mongodb://notifications_user:notifications_pass@localhost:27017/notifications_db?authSource=admin" --eval "db.system_notifications.find().pretty()"
```

```
[
  {
    _id: '4288fc70-8cd0-451a-b96e-b020ffcb0f04',
    request_id: 'f5257d92-2f0a-4512-b107-7f242e595ddc',
    service_name: 'device-service',
    reason: 'db_connection_failed',
    severity: 'critical',
    message: 'Could not connect to PostgreSQL',
    created_at: '2026-05-17T19:07:16.960617+00:00'
  }
]
```

Figura 28 — Documentos en MongoDB notifications_db

6.6 Redis — Tokens de Autenticación

Se confirmó que el refresh token del usuario fue correctamente añadido a la blacklist de Redis tras el logout de la Fase 1, impidiendo su reutilización.

```
PS C:\Users\luigi> docker exec -it ioteur-redis-auth-1 redis-cli KEYS "*"
1) "blacklist:eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0eXBlijoicmVmcVzaCisInN1IiI6ImE0YjMmNmZk1LTRiZDctNDM4Mj05YTA3LTYSNjMyMGNhNTI4YyIsImVtYVlsIjoiam9obk8leGFtcGxlLmVbSisInJvbmGU0iJ1c2VyIiwiaWF0IjoxNzc5MDQ0Mjc5LjE4IjE3Nzk2NDkwNzksImp8aSI6IjMmMGVlNmRhLWE2ODctNDQwYS1hMDNmLWFKOGZiNGUwMjAyMCJ9.skEmK5pvXKQPkSABZsGh17aTXKAUZWt3pqhVzNSYR1IietVLSdinTKFSQHApd35f4_WwDckrHrgbp0-vUF13AXQJfpaFLpQLIghBdjNNFnj6f318hy_1vh7ZgnZsBcrey-40F9_xEsW-NuDYRJjwz3Uebdt7N225IE3AEGjL98PKHTBeVGpemtunIk4EZb_g9wCZ-nA-ZwPgNAN_Hc4pETQaFaSan2vtuvHeuv0JEuR0ZGStcagJ244Ea06qa0_Brb78UPsiL_0a0ST10_H-cztyxqxUQoWcX2m0XaE9DUWmAoLU2WgdCXA_1D7J6SkFKohhOU10oLFvw0gPaTGHg"
```

Figura 29 — Keys en Redis auth (token blacklist)

6.7 Redis — Caché de Dispositivos

El dispositivo "Sensor Temp 01" se encuentra cacheado en Redis con todos sus atributos: UUID, MAC address, UUID del usuario propietario, estado activo y timestamp de último log.

```
PS C:\Users\luigi> docker exec -it ioteur-redis_device-1 redis-cli KEYS "*"
1) "device:1fe0e57e-9d4c-4e3d-b9a5-54213870c2a1"
PS C:\Users\luigi> docker exec -it ioteur-redis_device-1 redis-cli HGETALL "device:1fe0e57e-9d4c-4e3d-b9a5-54213870c2a1"
1) "device_uuid"
2) "1fe0e57e-9d4c-4e3d-b9a5-54213870c2a1"
3) "mac_address"
4) "AA:BB:CC:DD:EE:FF"
5) "device_name"
6) "Sensor Temp 01"
7) "user_uuid"
8) "a4b30795-4bd7-4382-9a07-696320ca528c"
9) "status"
10) "active"
11) "ultima_vez_log"
12) "2026-05-17T19:04:22.737800+00:00"
```

Figura 30 — Hash del dispositivo en Redis device cache

Errores Detectados por Mal Uso

Durante las pruebas se registraron errores derivados de un uso incorrecto de los endpoints. Estos errores son esperados y confirman que el sistema valida correctamente los tokens, permisos, IDs y formatos de parámetros.

```
api-gateway-1 | "GET /auth/me HTTP/1.1" 401 Unauthorized
```

→ Petición sin token de autenticación. El API Gateway rechaza correctamente la solicitud.

```
api-gateway-1 | "POST /auth/admin/register HTTP/1.1" 401 Unauthorized
```

→ Intento de registro de admin sin token con privilegios. Acceso denegado correctamente.

```
auth-service-1 | "PUT /auth/user/dla7738d-... HTTP/1.1" 404 Not Found
```

→ Intento de actualizar un usuario con un ID que no existe en la base de datos.

```
api-gateway-1 | "GET /reports/.../by-date?from=2025-01-01...&to=2025\12-31... HTTP/1.1" 422 Unprocessable Entity
```

→ Parámetro de fecha con formato incorrecto (backslash antes del guion). El servicio rechaza la solicitud con error 422 de validación.